



What is Amazon API Gateway?

Amazon API Gateway?

Amazon API Gateway is an AWS service for creating, publishing, maintaining, monitoring, and securing REST, HTTP, and WebSocket APIs at any scale. API developers can create APIs that access AWS or other web services, as well as data stored in the AWS Cloud. As an API Gateway API developer, you can create APIs for use in your own client applications. Or you can make your APIs available to third-party app developers.

API Gateway creates RESTful APIs that:

- Are HTTP-based.
- Enable stateless client-server communication.
- Implement standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

API Gateway creates WebSocket APIs that:

- Adhere to the WebSocket protocol, which enables stateful, full-duplex communication between client and server.
- Route incoming messages based on message content.

Architecture of API Gateway



Architecture of API Gateway

This diagram illustrates how the APIs you build in Amazon API Gateway provide you or your developer customers with an integrated and consistent developer experience for building AWS serverless applications. API Gateway handles all the tasks involved in accepting and processing up to hundreds of thousands of concurrent API calls. These tasks include traffic management, authorization and access control, monitoring, and API version management.

API Gateway acts as a "front door" for applications to access data, business logic, or functionality from your backend services, such as workloads running on Amazon Elastic Compute Cloud (Amazon EC2), code running on AWS Lambda, any web application, or real-time communication applications.

Features of API Gateway

Amazon API Gateway offers features such as the following:

- Support for stateful (WebSocket) and stateless (HTTP and REST) APIs.
- Powerful, flexible authentication mechanisms, such as AWS Identity and Access Management policies, Lambda authorizer functions, and Amazon Cognito user pools.
- Developer portal for publishing your APIs.
- Canary release deployments for safely rolling out changes.
- CloudTrail logging and monitoring of API usage and API changes.
- CloudWatch access logging and execution logging, including the ability to set alarms.
- Ability to use AWS CloudFormation templates to enable API creation.
- Support for custom domain names.
- Integration with AWS WAF for protecting your APIs against common web exploits.
- Integration with AWS X-Ray for understanding and triaging performance latencies.

Accessing API Gateway

You can access Amazon API Gateway in the following ways:

- AWS Management Console – The procedures throughout this guide explain how to use the AWS Management Console to perform tasks.
- AWS SDKs – If you're using a programming language that AWS provides an SDK for, you can use an SDK to access API Gateway. SDKs simplify authentication, integrate easily with your development environment, and provide access to API Gateway commands.
- API Gateway V1 and V2 APIs
- AWS Command Line Interface
- AWS Tools for Windows PowerShell

Part of AWS serverless infrastructure

Together with AWS Lambda, API Gateway forms the app-facing part of the AWS serverless infrastructure.

For an app to call publicly available AWS services, you can use Lambda to interact with required services and expose Lambda functions through API methods in API Gateway. AWS Lambda runs your code on a highly available computing infrastructure. It performs the necessary execution and administration of computing resources. To enable serverless applications, API Gateway supports streamlined proxy integrations with AWS Lambda and HTTP endpoints.



API Gateway use cases

Use API Gateway to create HTTP APIs

HTTP APIs enable you to create RESTful APIs with lower latency and lower cost than REST APIs.

You can use HTTP APIs to send requests to AWS Lambda functions or to any publicly routable HTTP endpoint.

For example, you can create an HTTP API that integrates with a Lambda function on the backend. When a client calls your API, API Gateway sends the request to the Lambda function and returns the function's response to the client.

HTTP APIs support **OpenID Connect** and **OAuth 2.0** authorization. They come with built-in support for cross-origin resource sharing (CORS) and automatic deployments.

Use API Gateway to create REST APIs

An API Gateway REST API is made up of resources and methods. A resource is a logical entity that an app can access through a resource path. A method corresponds to a REST API request that is submitted by the user of your API and the response returned to the user.

For example, `/incomes` could be the path of a resource representing the income of the app user. A resource can have one or more operations that are defined by appropriate HTTP verbs such as GET, POST, PUT, PATCH, and DELETE. A combination of a resource path and an operation identifies a method of the API. For example, a `POST /incomes` method could add an income earned by the caller, and a `GET /expenses` method could query the reported expenses incurred by the caller.

The app doesn't need to know where the requested data is stored and fetched from on the backend. In API Gateway REST APIs, the frontend is encapsulated by *method requests* and *method responses*. The API interfaces with the backend by means of *integration requests* and *integration responses*.

Use API Gateway to create REST APIs

For example, with DynamoDB as the backend, the API developer sets up the integration request to forward the incoming method request to the chosen backend. The setup includes specifications of an appropriate DynamoDB action, required IAM role and policies, and required input data transformation. The backend returns the result to API Gateway as an integration response.

To route the integration response to an appropriate method response (of a given HTTP status code) to the client, you can configure the integration response to map required response parameters from integration to method. You then translate the output data format of the backend to that of the frontend, if necessary. API Gateway enables you to define a schema or model for the payload to facilitate setting up the body mapping template.

API Gateway provides REST API management functionality such as the following:

- Support for generating SDKs and creating API documentation using API Gateway extensions to OpenAPI
- Throttling of HTTP requests

Use API Gateway to create WebSocket

API

In a WebSocket API, the client and the server can both send messages to each other at any time. Backend servers can easily push data to connected users and devices, avoiding the need to implement complex polling mechanisms.

For example, you could build a serverless application using an API Gateway WebSocket API and AWS Lambda to send and receive messages to and from individual users or groups of users in a chat room. Or you could invoke backend services such as AWS Lambda, Amazon Kinesis, or an HTTP endpoint based on message content.

You can use API Gateway WebSocket APIs to build secure, real-time communication applications without having to provision or manage any servers to manage connections or large-scale data exchanges. Targeted use cases include real-time applications such as the following:

- Chat applications
- Real-time dashboards such as stock tickers
- Real-time alerts and notifications

Use API Gateway to create WebSocket

API

API Gateway provides WebSocket API management functionality such as the following:

- Monitoring and throttling of connections and messages
- Using AWS X-Ray to trace messages as they travel through the APIs to backend services
- Easy integration with HTTP/HTTPS endpoints

Who uses API Gateway?

There are two kinds of developers who use API Gateway: API developers and app developers.

An API developer creates and deploys an API to enable the required functionality in API Gateway. The API developer must be an IAM user in the AWS account that owns the API.

An app developer builds a functioning application to call AWS services by invoking a WebSocket or REST API created by an API developer in API Gateway.

The app developer is the customer of the API developer. The app developer doesn't need to have an AWS account, provided that the API either doesn't require IAM permissions or supports authorization of users through third-party federated identity providers supported by Amazon Cognito user pool identity federation. Such identity providers include Amazon, Amazon Cognito user pools, Facebook, and Google.



Amazon API Gateway concepts

API Gateway concepts

API Gateway

API Gateway is an AWS service that supports the following:

- Creating, deploying, and managing a [RESTful](#) application programming interface (API) to expose backend HTTP endpoints, AWS Lambda functions, or other AWS services.
- Creating, deploying, and managing a [WebSocket](#) API to expose AWS Lambda functions or other AWS services.
- Invoking exposed API methods through the frontend HTTP and WebSocket endpoints.

API Gateway REST API

A collection of HTTP resources and methods that are integrated with backend HTTP endpoints, Lambda functions, or other AWS services. You can deploy this collection in one or more stages. Typically, API resources are organized in a resource tree according to the application logic. Each API resource can expose one or more API methods that have unique HTTP verbs supported by API Gateway.

API Gateway HTTP API

A collection of routes and methods that are integrated with backend HTTP endpoints or Lambda functions. You can deploy this collection in one or more stages. Each route can expose one or more API methods that have unique HTTP verbs supported by API Gateway.

API Gateway concepts

API Gateway WebSocket API

A collection of WebSocket routes and route keys that are integrated with backend HTTP endpoints, Lambda functions, or other AWS services. You can deploy this collection in one or more stages. API methods are invoked through frontend WebSocket connections that you can associate with a registered custom domain name.

API deployment

A point-in-time snapshot of your API Gateway API. To be available for clients to use, the deployment must be associated with one or more API stages.

API developer

Your AWS account that owns an API Gateway deployment (for example, a service provider that also supports programmatic access).

API endpoint

A hostname for an API in API Gateway that is deployed to a specific Region. The hostname is of the form `{api-id}.execute-api.{region}.amazonaws.com`. The following types of API endpoints are supported:

- Edge-optimized API endpoint
- Private API endpoint
- Regional API endpoint

API Gateway concepts

API key

An alphanumeric string that API Gateway uses to identify an app developer who uses your REST or WebSocket API. API Gateway can generate API keys on your behalf, or you can import them from a CSV file. You can use API keys together with Lambda authorizers or usage plans to control access to your APIs.

API owner

API stage

A logical reference to a lifecycle state of your API (for example, 'dev', 'prod', 'beta', 'v2'). API stages are identified by API ID and stage name.

App developer

An app creator who may or may not have an AWS account and interacts with the API that you, the API developer, have deployed. App developers are your customers. An app developer is typically identified by an API key.

Callback URL

When a new client is connected to through a WebSocket connection, you can call an integration in API Gateway to store the client's callback URL. You can then use that callback URL to send messages to the client from the backend system.

API Gateway concepts

Developer portal

An application that allows your customers to register, discover, and subscribe to your API products (API Gateway usage plans), manage their API keys, and view their usage metrics for your APIs.

Edge-optimized API endpoint

The default hostname of an API Gateway API that is deployed to the specified Region while using a CloudFront distribution to facilitate client access typically from across AWS Regions. API requests are routed to the nearest CloudFront Point of Presence (POP), which typically improves connection time for geographically diverse clients.

Integration request

The internal interface of a WebSocket API route or REST API method in API Gateway, in which you map the body of a route request or the parameters and body of a method request to the formats required by the backend.

Integration response

The internal interface of a WebSocket API route or REST API method in API Gateway, in which you map the status codes, headers, and payload that are received from the backend to the response format that is returned to a client app.

API Gateway concepts

Mapping template

A script in Velocity Template Language (VTL) that transforms a request body from the frontend data format to the backend data format, or that transforms a response body from the backend data format to the frontend data format. Mapping templates can be specified in the integration request or in the integration response.

They can reference data made available at runtime as context and stage variables.

The mapping can be as simple as an identity transform that passes the headers or body through the integration as-is from the client to the backend for a request. The same is true for a response, in which the payload is passed from the backend to the client.

Method request

The public interface of an API method in API Gateway that defines the parameters and body that an app developer must send in requests to access the backend through the API.

Method response

The public interface of a REST API that defines the status codes, headers, and body models that an app developer should expect in responses from the API.

API Gateway concepts

Mock integration

In a mock integration, API responses are generated from API Gateway directly, without the need for an integration backend. As an API developer, you decide how API Gateway responds to a mock integration request. For this, you configure the method's integration request and integration response to associate a response with a given status code.

Model

A data schema specifying the data structure of a request or response payload. A model is required for generating a strongly typed SDK of an API. It is also used to validate payloads. A model is convenient for generating a sample mapping template to initiate creation of a production mapping template. Although useful, a model is not required for creating a mapping template.

Private API

API Gateway concepts

Private API endpoint

An API endpoint that is exposed through interface VPC endpoints and allows a client to securely access private API resources inside a VPC. Private APIs are isolated from the public internet, and they can only be accessed using VPC endpoints for API Gateway that have been granted access.

Private integration

An API Gateway integration type for a client to access resources inside a customer's VPC through a private REST API endpoint without exposing the resources to the public internet.

Proxy integration

A simplified API Gateway integration configuration. You can set up a proxy integration as an HTTP proxy integration or a Lambda proxy integration.

For HTTP proxy integration, API Gateway passes the entire request and response between the frontend and an HTTP backend. For Lambda proxy integration, API Gateway sends the entire request as input to a backend Lambda function. API Gateway then transforms the Lambda function output to a frontend HTTP response. In REST APIs, proxy integration is most commonly used with a proxy resource, which is represented by a greedy path variable (for example, {proxy+}) combined with a catch-all ANY method.

API Gateway concepts

Quick create

You can use quick create to simplify creating an HTTP API. Quick create creates an API with a Lambda or HTTP integration, a default catch-all route, and a default stage that is configured to automatically deploy changes.

Regional API endpoint

The host name of an API that is deployed to the specified Region and intended to serve clients, such as EC2 instances, in the same AWS Region. API requests are targeted directly to the Region-specific API Gateway API without going through any CloudFront distribution. For in-Region requests, a Regional endpoint bypasses the unnecessary round trip to a CloudFront distribution.

In addition, you can apply latency-based routing on Regional endpoints to deploy an API to multiple Regions using the same Regional API endpoint configuration, set the same custom domain name for each deployed API, and configure latency-based DNS records in Route 53 to route client requests to the Region that has the lowest latency.

API Gateway concepts

Route

A WebSocket route in API Gateway is used to direct incoming messages to a specific integration, such as an AWS Lambda function, based on the content of the message. When you define your WebSocket API, you specify a route key and an integration backend. The route key is an attribute in the message body. When the route key is matched in an incoming message, the integration backend is invoked.

A default route can also be set for non-matching route keys or to specify a proxy model that passes the message through as-is to backend components that perform the routing and process the request.

Route request

The public interface of a WebSocket API method in API Gateway that defines the body that an app developer must send in the requests to access the backend through the API.

Route response

The public interface of a WebSocket API that defines the status codes, headers, and body models that an app developer should expect from API Gateway.

API Gateway concepts

Usage plan

A usage plan provides selected API clients with access to one or more deployed REST or WebSocket APIs. You can use a usage plan to configure throttling and quota limits, which are enforced on individual client API keys.

WebSocket connection

API Gateway maintains a persistent connection between clients and API Gateway itself. There is no persistent connection between API Gateway and backend integrations such as Lambda functions. Backend services are invoked as needed, based on the content of messages received from clients.

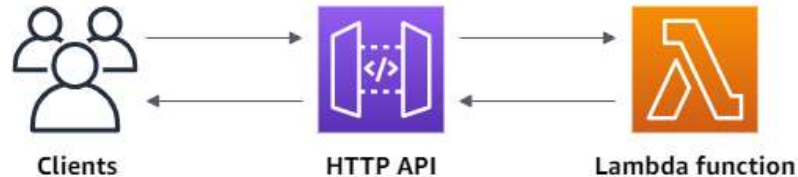


Getting started with API Gateway

Getting started with API Gateway

First, you create a Lambda function using the AWS Lambda console. Next, you create an HTTP API using the API Gateway console. Then, you invoke your API.

When you invoke your HTTP API, API Gateway routes the request to your Lambda function. Lambda runs the Lambda function and returns a response to API Gateway. API Gateway then returns a response to you.



To complete this exercise, you need an AWS account and an AWS Identity and Access Management user with console access.

Step 1: Create a Lambda function

You use a Lambda function for the backend of your API. Lambda runs your code only when needed and scales automatically, from a few requests per day to thousands per second.

For this example, you use the default Node.js function from the Lambda console.

To create a Lambda function

1. Sign in to the Lambda console at <https://console.aws.amazon.com/lambda>.
2. Choose **Create function**.
3. For **Function name**, enter **my-function**.
4. Choose **Create function**.

Step 1: Create a Lambda function

The example function returns a 200 response to clients, and the text Hello from Lambda!.

You can modify your Lambda function, as long as the function's response aligns with the format that API Gateway requires.

The default Lambda function code should look similar to the following:

```
exports.handler = async (event) => {  
  const response = {  
    statusCode: 200,  
    body: JSON.stringify('Hello from Lambda!'),  
  };  
  return response;  
};
```

Step 2: Create an HTTP API

Next, you create an HTTP API. API Gateway also supports REST APIs and WebSocket APIs, but an HTTP API is the best choice for this exercise. HTTP APIs have lower latency and lower cost than REST APIs. WebSocket APIs maintain persistent connections with clients for full-duplex communication, which isn't required for this example. The HTTP API provides an HTTP endpoint for your Lambda function. API Gateway routes requests to your Lambda function, and then returns the function's response to clients.

To create an HTTP API

1. Sign in to the API Gateway console at <https://console.aws.amazon.com/apigateway>.
2. Do one of the following:
 - To create your first API, for **HTTP API**, choose **Build**.
 - If you've created an API before, choose **Create API**, and then choose **Build** for **HTTP API**.

Step 2: Create an HTTP API

3. For **Integrations**, choose **Add integration**.
4. Choose **Lambda**.
5. For **Lambda function**, enter **my-function**.
6. For **API name**, enter **my-http-api**.
7. Choose **Next**.
8. Review the *route* that API Gateway creates for you, and then choose **Next**.
9. Review the *stage* that API Gateway creates for you, and then choose **Next**.
10. Choose **Create**.

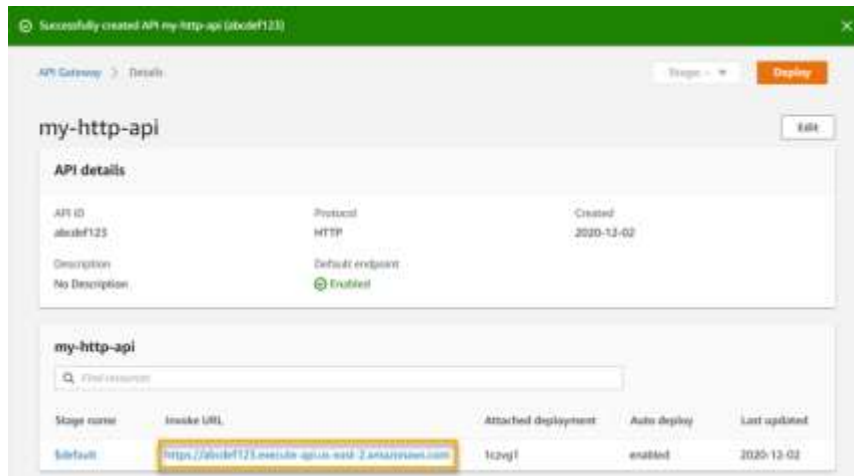
Now you've created an HTTP API with a Lambda integration that's ready to receive requests from clients.

Step 3: Test your API

Next, you test your API to make sure that it's working. For simplicity, use a web browser to invoke your API.

To test your API

1. Sign in to the API Gateway console at <https://console.aws.amazon.com/apigateway>.
2. Choose your API.
3. Note your API's invoke URL.



Step 3: Test your API

4. Copy your API's invoke URL, and enter it in a web browser. Append the name of your Lambda function to your invoke URL to call your Lambda function. By default, the API Gateway console creates a route with the same name as your Lambda function, `my-function`.

The full URL should look like `https://abcdef123.execute-api.us-east-2.amazonaws.com/my-function`.

Your browser sends a `GET` request to the API.

5. Verify your API's response. You should see the text `"Hello from Lambda!"` in your browser.

(Optional) Step 4: Clean up

To prevent unnecessary costs, delete the resources that you created as part of this getting started exercise. The following steps delete your HTTP API, your Lambda function, and associated resources.

To delete an HTTP API

1. Sign in to the API Gateway console at <https://console.aws.amazon.com/apigateway>.
2. On the **APIs** page, select an API. Choose **Actions**, and then choose **Delete**.
3. Choose **Delete**.

To delete a Lambda function

1. Sign in to the Lambda console at <https://console.aws.amazon.com/lambda>.
2. On the **Functions** page, select a function. Choose **Actions**, and then choose **Delete**.
3. Choose **Delete**.

(Optional) Step 4: Clean up

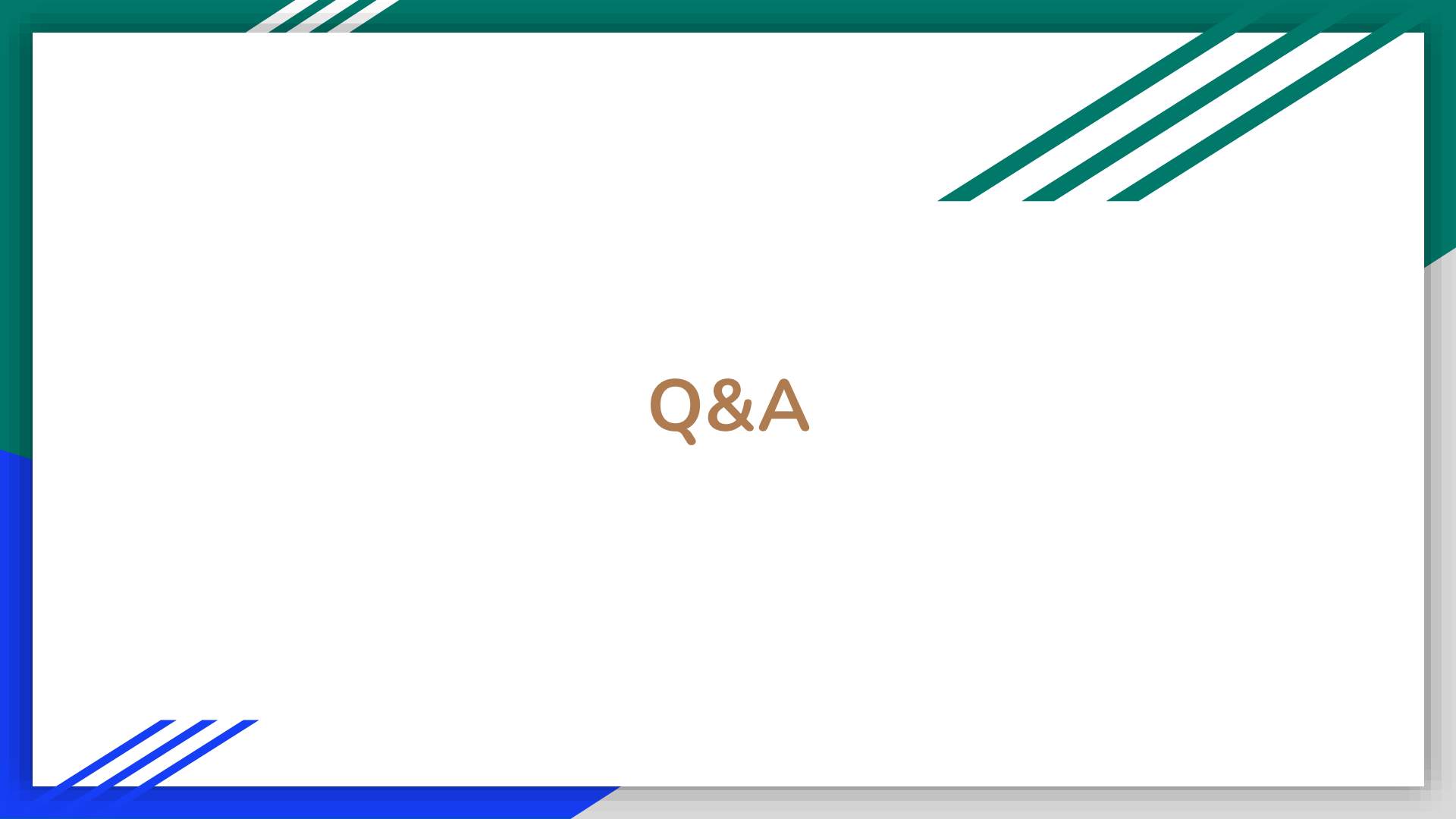
To delete a Lambda function's log group

1. In the Amazon CloudWatch console, open the **Log groups** page.
2. On the **Log groups** page, select the function's log group (`/aws/lambda/my-function`). Choose **Actions**, and then choose **Delete log group**.
3. Choose **Delete**.

To delete a Lambda function's execution role

1. In the AWS Identity and Access Management console, open the **Roles** page.
2. Select the function's role, for example, `my-function-31exxmpl`.
3. Choose **Delete role**.
4. Choose **Yes, delete**.

You can automate the creation and cleanup of AWS resources by using AWS CloudFormation or AWS SAM.



Q&A